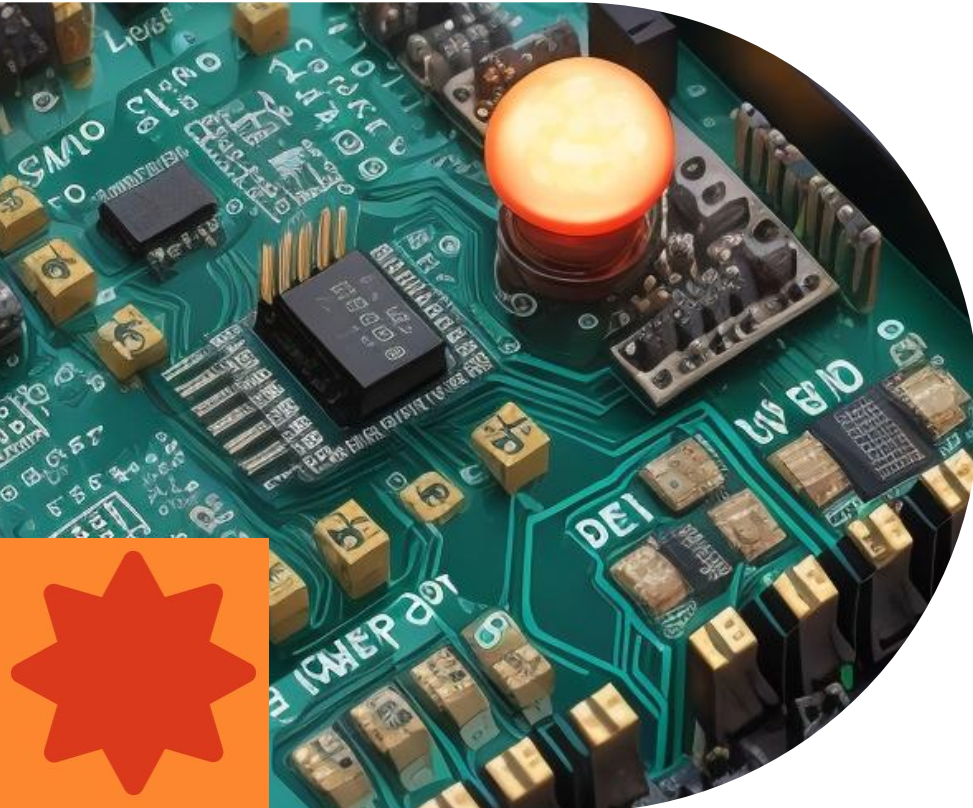


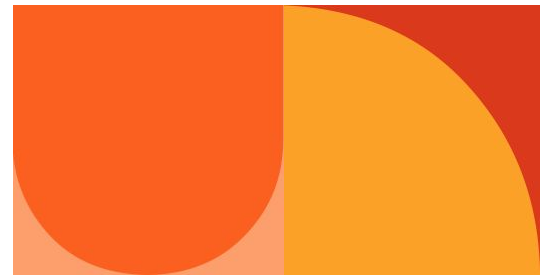


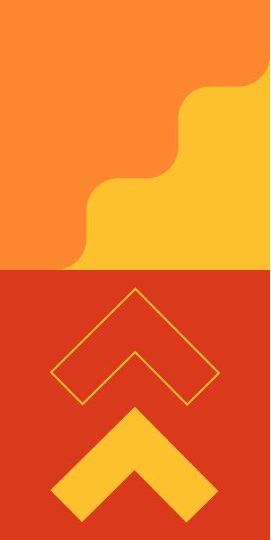
C . E . S . A . R



# Sistemas Digitais

## Aula 08: Saídas Digitais





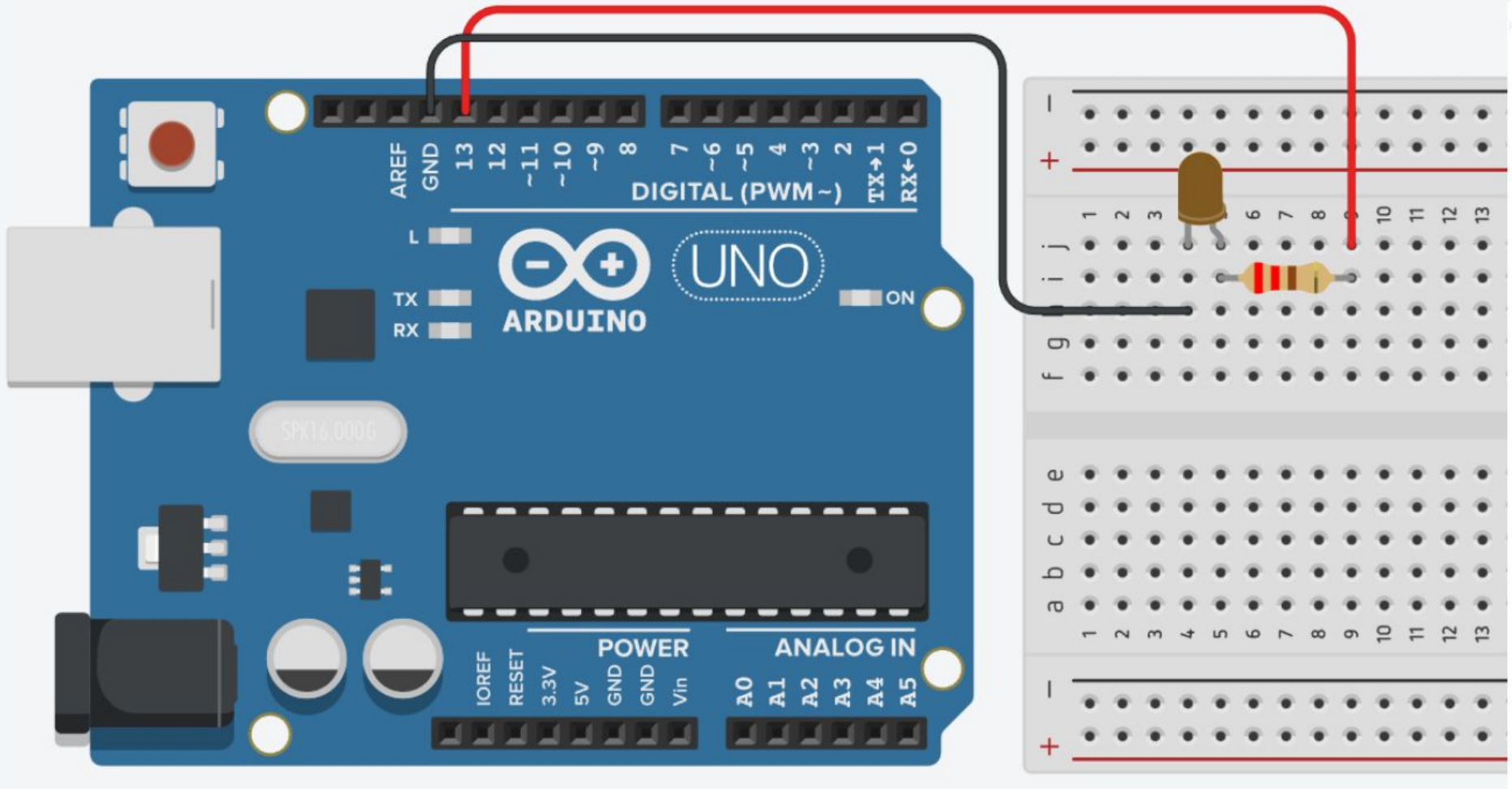
# Nos capítulos anteriores de Sistemas Digitais...

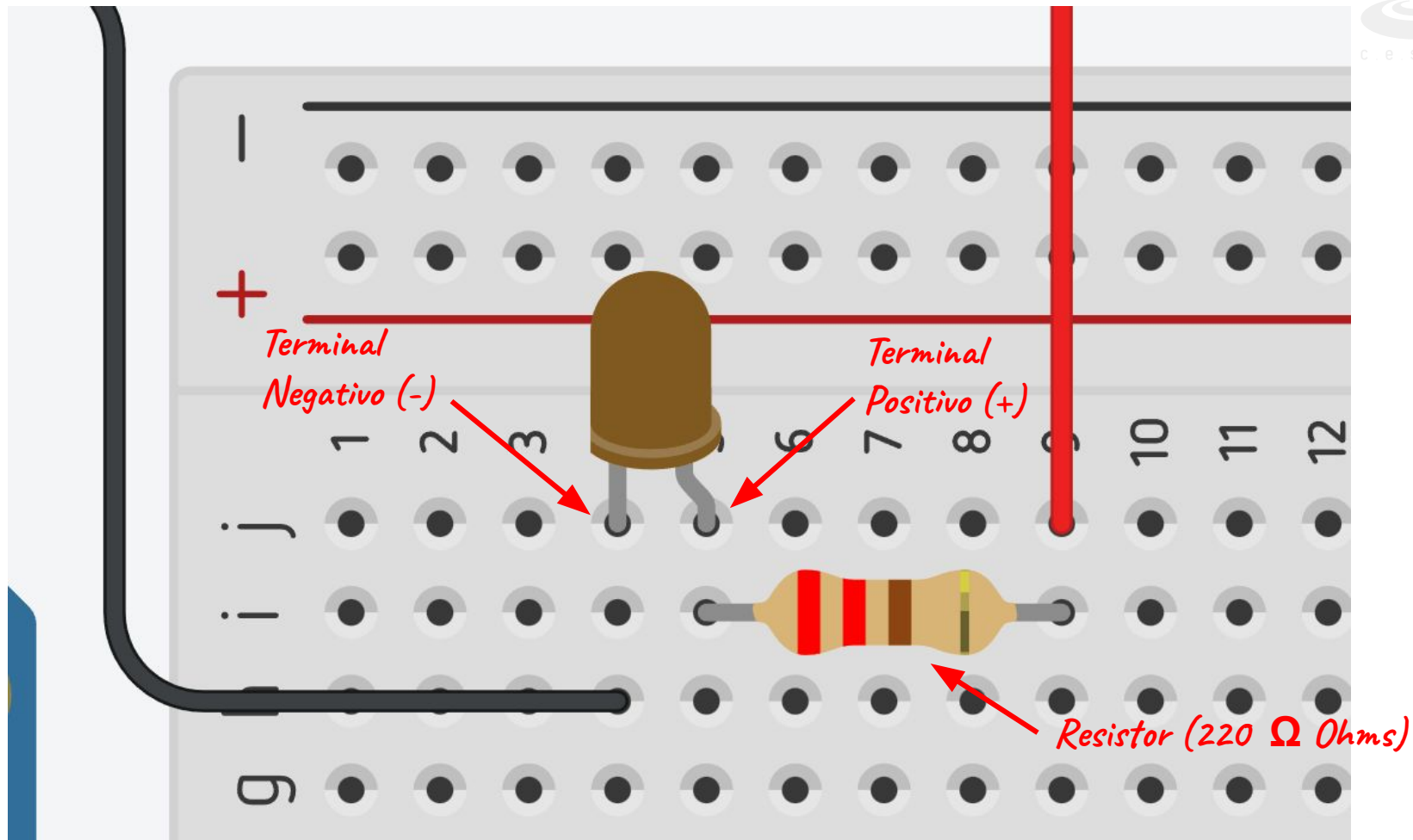
Básico sobre circuitos:

- Arduino;
- Resistor;
- Protoboard;
- LED;
- Jumper.

Nosso primeiro código:

- `setup()` e `loop()`;
- `pinMode(pino, modo)`;
- `digitalWrite(pino, valor)`;
- `delay(milisegundos)`

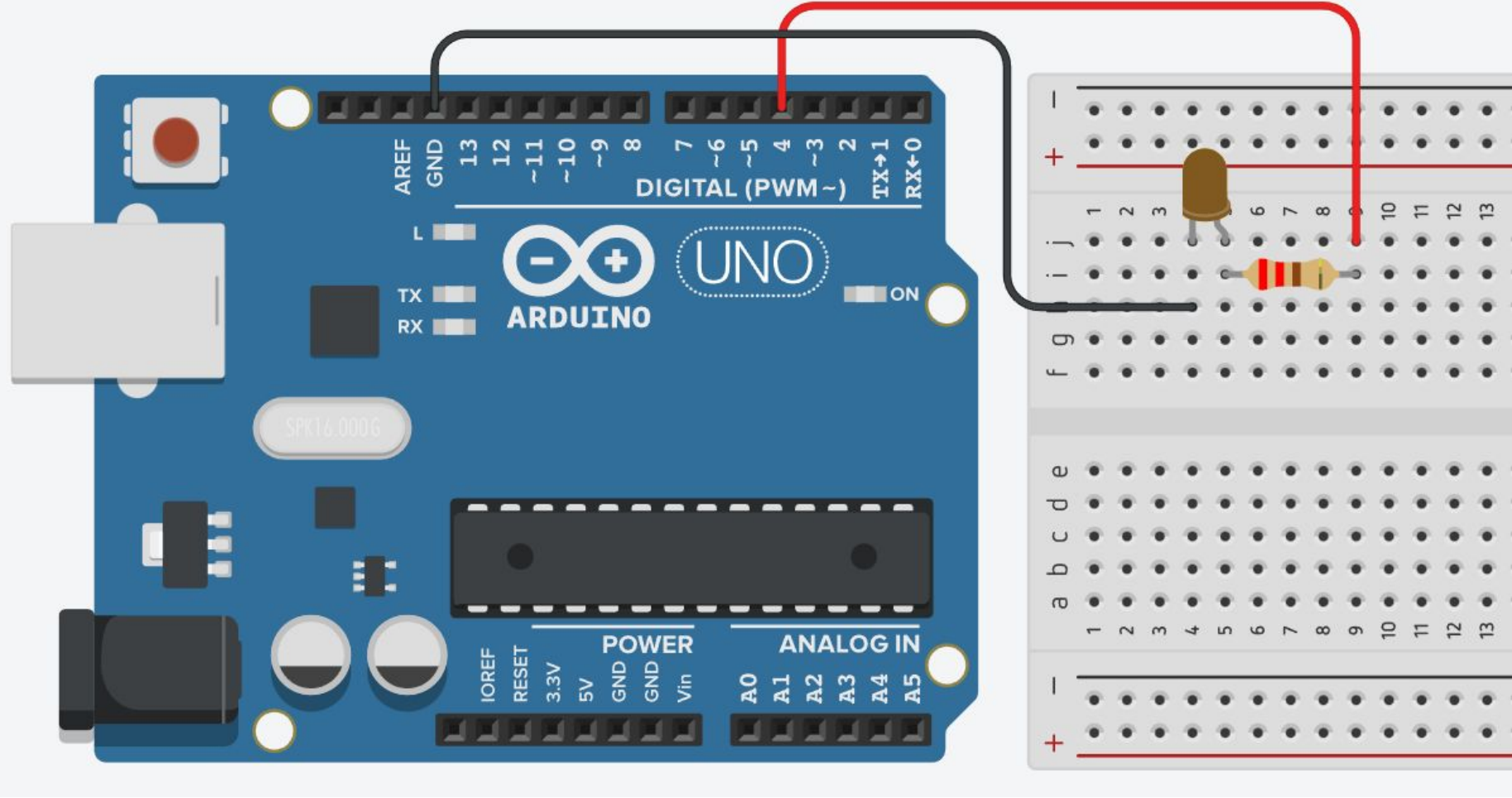




# Código



```
const int LED_PIN = 13;
void setup() {
  pinMode(LED_PIN, OUTPUT);
}
void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(1000); // milissegundos
  digitalWrite(LED_PIN, LOW);
  delay(1000); // milissegundos
}
```





# Código



```
const int LED_PIN = 4;
void setup() {
  pinMode(LED_PIN, OUTPUT);
}
void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(1000); // milissegundos
  digitalWrite(LED_PIN, LOW);
  delay(1000); // milissegundos
}
```

# Programando em Arduino

Os códigos em Arduino são chamados de **Sketchs**.

- Um **programa/código** que você escreve
- Usado para controlar placas como o Arduino Uno

Para escrever o código, usamos a [Arduino IDE](#) (versão 2).

Caso seja necessário, instale os drivers disponíveis neste [link](#) (eles também estão disponíveis através do portal de autoatendimento nos computadores da CESAR School).





# O básico de um sketch

Todo código em Arduino tem essas duas funções básicas

## setup()

É onde colocamos os códigos de configuração e inicialização. Roda apenas **UMA** vez, quando o Arduino é ligado.

## loop()

É onde colocamos os códigos que executam os atuadores e as leituras dos sensores. Roda repetidamente (**loop**).



```
void setup() {  
  // executa UMA VEZ quando a placa liga ou é resetada  
}  
  
void loop() {  
  // executa SEM PARAR, milhares de vezes por segundo  
}
```

# Estrutura

Tudo que precisa acontecer só uma vez

```
void setup() {  
    // executa UMA VEZ quando a placa liga ou é resetada  
}  
void loop() {  
    // executa SEM PARAR, milhares de vezes por segundo  
}
```

Tudo que precisa acontecer continuamente



# Os comandos que aprendemos

`pinMode(pino, modo)`

Essa função configura um pino do Arduino como entrada (INPUT) ou saída (OUTPUT).

`digitalWrite(pino, valor)`

Aciona um valor de **ligado** (HIGH) ou **desligado** (LOW) em um pino digital do Arduino. Para funcionar corretamente, esse pino deve ter sido configurado como OUTPUT anteriormente.

`delay(milisegundos)`

Pausa o programa por uma quantidade especificada de tempo (em milissegundos).





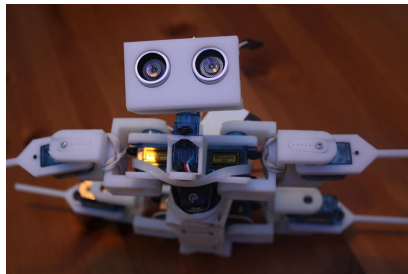
# Variáveis

# Variáveis

Quando escrevemos um código, muitas vezes precisamos guardar informações para usar depois.

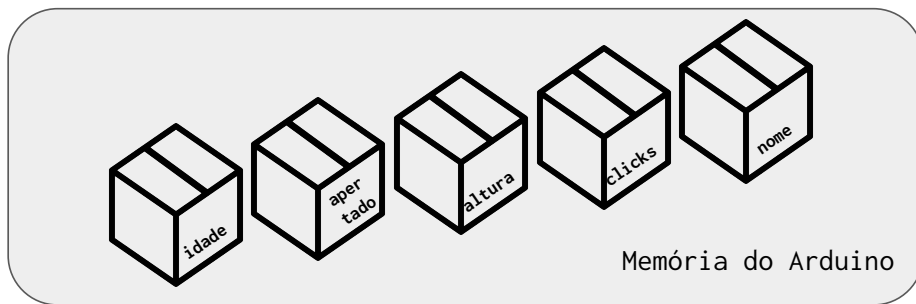
Pode ser o valor de um sensor, a quantidade de vezes que alguém apertou um botão ou qualquer outra coisa importante. Para fazer isso, usamos algo chamado "**variáveis**".

Variáveis são como caixinhas na memória onde podemos guardar dados.



# Variáveis

É como colocar um nome na caixinha para sabermos o que tem dentro dela.



É como colocar um nome na caixinha para sabermos o que tem dentro dela.

Por exemplo, podemos ter uma caixinha chamada "idade" e colocar a idade da pessoa lá dentro. Isso nos ajuda a lembrar e usar essa informação mais tarde no nosso programa.

# Tipos de variáveis

As variáveis podem ser de diferentes tipos:

*Númericos:*

**int** - números inteiros (1, 2, 3, -8, -50)

**float** - números reais (1.0, 5.5, -32.45)

*Lógicos:*

**bool** - valor lógico de verdadeiro (true) ou falso (false)

*Texto:*

**char** - representação de um caractere ('a', 'd', 'x', '?', '7')

**String** - sequência de caracteres ("Eita", "Oxe", "Vish")

Mais sobre os tipos de dados em Arduino [aqui](#).



# Declarando variáveis

Para criar uma variável no código, é necessário colocar o seu **tipo** e um **nome**.

```
1 | int idade;
2 | float altura = 1.83;
3 |
4 |
5 |
... |
10 |
11 | idade = 23;
```

*Declaração*

*Tipo*

*Nome*

*Valor*

*Atribuição*

# Declarando variáveis

Para criar uma variável no código, é necessário colocar o seu **tipo** e um **nome**.

```
1 | int ledPin = 4;
2 |
3 | void setup() {
4 |     pinMode(ledPin, OUTPUT);           // configura o pino digital 4 como saída
5 | }
6 |
7 | void loop() {
8 |     digitalWrite(ledPin, HIGH);        // ativa o pino digital 4
9 |     delay(1000);                       // espera por um segundo
10 |    digitalWrite(ledPin, LOW);          // desativa o pino digital 4
11 |    delay(1000);                       // espera por um segundo
12 | }
```

# Constantes

Caso seja necessário, também é possível restringir o acesso a modificação do valor de uma variável, tornando-a uma constante:

```
const int ledPin = 4;
```

Neste caso, é obrigatório que, além da declaração da variável, também haja a atribuição de um valor.



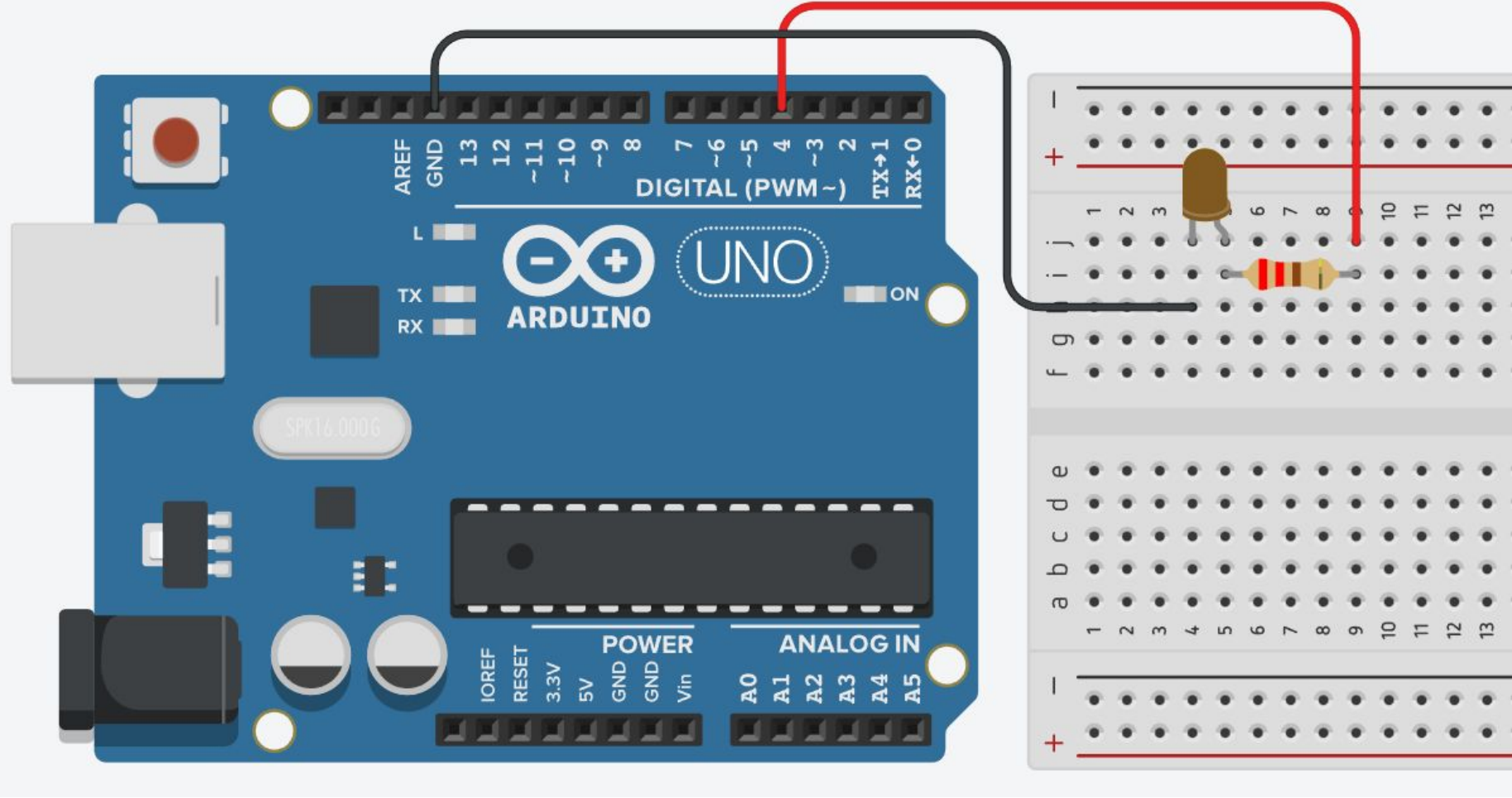
# LED Challenge

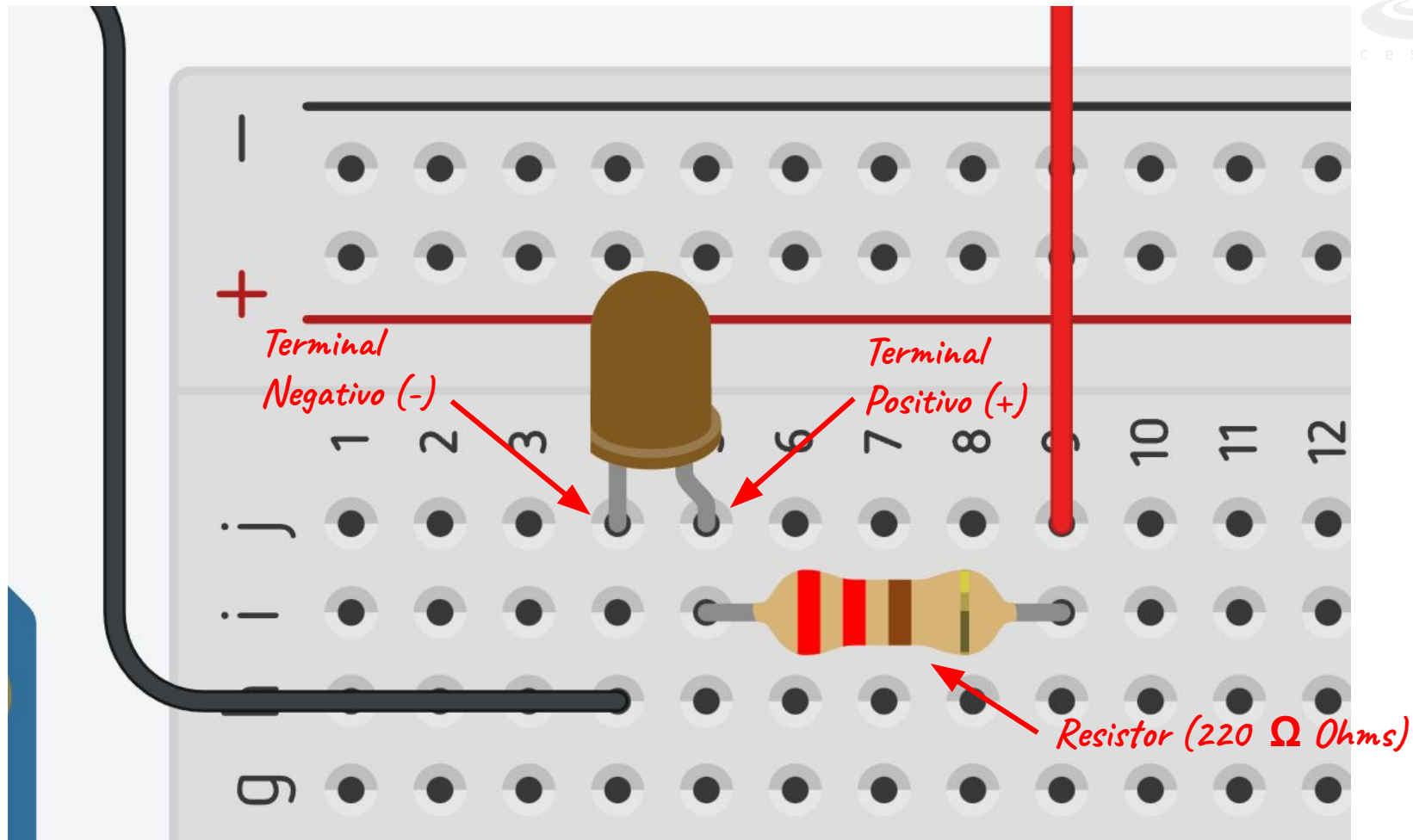


# Para não esquecer:

- Todo comando termina com ponto e vírgula (;)
- Comentários podem ser feitos com //
- Apesar de não ser obrigatório, indentar o código é uma boa prática
- **Tudo** o que é preciso **saber** sobre **programar** em **Arduino** você **encontra** [aqui](#).







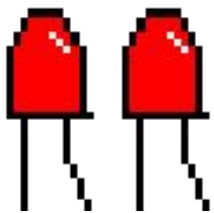


```
1 | const int ledPin = 4;
2 |
3 | void setup() {
4 |     pinMode(ledPin, OUTPUT);           // configura o pino digital 4 como saída
5 | }
6 |
7 | void loop() {
8 |     digitalWrite(ledPin, HIGH);        // ativa o pino digital 4
9 |     delay(1000);                       // espera por um segundo
10 |    digitalWrite(ledPin, LOW);          // desativa o pino digital 4
11 |    delay(1000);                       // espera por um segundo
12 | }
```

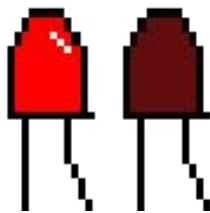


# LED Challenge

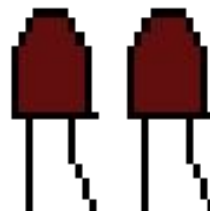
- Adicione ao circuito da aula anterior mais uma LED;
- Faça cada um desses exercícios em um sketch separado;
- Salve cada sketch.



1. Leds síncronas



2. Leds alternadas



3. Leds ritmadas



---

## Extra:

- Experimente outros componentes digitais no lugar da LED;
- Use variáveis;
- 🌟 Acenda uma LED quando um botão for pressionado.



C . E . S . A . R

Pessoas impulsionando inovação.  
Inovação impulsionando negócios.

